

SERVICE MANAGEMENT DASHBOARD

Documento Técnico

Arquitetura, Implementação e Guia Completo de Replicação — v2.0

Marco 2026 | Arrocha | AI Studio + Antigravity

1. Arquitetura da Solucao

O SMD e um Single Page Application (SPA) totalmente client-side. Nao existe backend, servidor web, banco de dados externo ou API calls em tempo de execucao no browser. Toda a logica, dados e visualizacoes estao em um unico arquivo HTML de ~1.7MB.

1.1 Stack Tecnologica

Tecnologia	Versao	Papel no Projeto
HTML5 + CSS3	Nativo	Estrutura, estilos e animacoes (sla-pulse, etc.)
JavaScript ES5/ES6	Nativo	63 funcoes de logica, filtros, graficos, modais
Chart.js	CDN 3.x	Graficos de barras, linhas. SVG para gauges SLA.
Python 3.8+	3.8+	smd_agent.py: processamento CSV + orquestracao IA
pandas	1.3+	Leitura e transformacao do Tickets.csv
Google Gemini API	gemini-2.0-flash	6 agentes IA especializados (gratuito)
urllib.request	stdlib Python	HTTP calls para API Gemini (sem dependencias extras)
logging.handlers	stdlib Python	RotatingFileHandler: 500KB x 3 arquivos

1.2 Estrutura de Arquivos

Caminho	Tipo	Descricao
C:/Dashboard/arrocha_dashboard_v6.html	HTML	Dashboard principal. Contem todos dados + logica.
C:/Dashboard/input/Tickets.csv	CSV	Fonte de dados. Sep=; Encoding=UTF-8-BOM.
C:/Dashboard/smd_agent.py	Python	Orquestrador dos 6 agentes Gemini.
C:/Dashboard/api_key.txt	Texto	Chave Gemini (linha iniciando com Alza).
C:/Dashboard/smd_agent.log	Log	Log rotativo do agente.
C:/Dashboard/Resultados/operations_output.json	JSON	Output: Operations Advisor
C:/Dashboard/Resultados/predictive_output.json	JSON	Output: Predictive Analyst
C:/Dashboard/Resultados/improvement_output.json	JSON	Output: Improvement Designer
C:/Dashboard/Resultados/market_output.json	JSON	Output: Market Analyst
C:/Dashboard/Resultados/qa_output.json	JSON	Output: QA Tester
C:/Dashboard/Resultados/triage_output.json	JSON	Output: Triage Analyst

2. Formato de Dados

2.1 Especificação do CSV de Entrada

O Tickets.csv deve usar separador ponto-e-virgula (;) e encoding UTF-8 com BOM. Colunas obrigatórias:

Coluna	Tipo	Descrição	Exemplos
Project Name	String	Nome do projeto/cliente	Chanel, MAKRO (PT) CMS 2025
Ticket	Integer	ID numerico unico	108701, 108702
Priority	String	Urgencia do ticket	P1, P2, P3, P4
Severity	String	Tipo do ticket	incident, user_request, problem, change_request, internal
Status	String	Estado atual	acknowledged, assigned_for_analysis, closed, resolved
Opening Date	Date	Data de abertura	YYYY-MM-DD ou DD/MM/YYYY
Close Date	Date	Data de fechamento	Vazio se aberto
Last Updated Date	Date	Ultima atualizacao	YYYY-MM-DD HH:MM
Resolution SLA	Float	Minutos desde abertura ate fechamento	360 = no SLA, -120 = fora
Application	String	Sistema relacionado	XStore, Middleware, SAP
Environment	String	Ambiente do ticket	PRD, HML, DEV
Root Cause Source	String	Grupo de causa raiz	Client, RC, Oracle, Problem Analysis
Root Cause Type	String	Tipo de causa raiz	Bug, Infrastructure, Business Process
Summary	String	Título/resumo do ticket	Texto livre
assigned	String	Login do responsavel	joao.silva, maria.santos

2.2 Formato Compacto Interno (Array-of-Arrays)

Para performance com milhares de tickets, dados são serializados em formato compacto no HTML. O JavaScript usa 3 estruturas globais:

```
// Campos (ordem fixa)
var _TF =
['k','eid','pr','sv','st','op','res','cl','ap','en','su','upd','ass','sl','rc','rct',
'rs','prj','y_o','m_o','d_o','y_c','m_c','d_c'];
// Rows (uma por ticket, valores na ordem de _TF)
var _ROWS = [[108701,"","P1","incident","closed","2026-01-15",...], ...];
// Indice para lookup O(1) por ticket ID
var _IDX = {"108701": 0, "108702": 1, ...};
```

A função `getTix(key)` converte `String(key)` e faz lookup via `_IDX`, retornando objeto com propriedades nomeadas. Resultados memoizados em `_MEMO`.

3. Funcoes JavaScript Criticas

O dashboard tem 63 funcoes JavaScript. As mais criticas para manutencao e extensao sao:

Funcao	Responsabilidade	Dependencias	Notas
getTix(key)	Lookup ticket por ID. O(1) via _IDX + memoizacao _MEMO.	_ROWS, _IDX, _TF, _MEMO	String(key) obrigatorio — _IDX tem chaves string
getSLAFiltered()	Filtra D.sla por selYear, selMonth e selProject.	D.sla, getTix, selProject	Usa getTix(t.tid).prj para filtro por projeto
getMonthCounts(sev,k)	Retorna {opened, closed, o_ids, c_ids} para um mes.	D.monthly, filterByProject	Aplica filtro de projeto via filterByProject()
compCount(sev,y,m,w)	Conta tickets abertos/fechados num mes respeitando projeto.	D.monthly, getTix	GLOBAL — nao pode ser closure de panelHTML
buildSLAMini(cid)	Renderiza 4 gauges SVG com badge urgencia se pct < 80%.	getSLAFiltered, CFG.sla	Usa var "out" (nao "html") para evitar conflito
buildAssigned(sev,cid)	Gera grafico barras horizontais por responsavel.	_ROWS, ASSIGNED_STATUS	Escaneia _ROWS diretamente — sem filtro ano/mes
buildHoje()	Monta aba Hoje com tickets do dia + previsao.	_ROWS, _TF, AI_INSIGHTS	Usa new Date() do browser para dia corrente
buildHojeForecast()	Grafico historico 14 dias + projecao 4 semanas.	_ROWS, AI_INSIGHTS.predictive	Fallback para media se weekly_forecast ausente
buildAttention()	Lista alertas + SLA urgentes calculados em runtime.	getSLAFiltered, D.attention	SLA alerts somam antes dos estaticos do Python
buildRange()	Calcula volume e exibe barra de esforco Man-Days.	getMonthWin, getMonthCounts	Requer selProject ou selMonth para exibir dados
renderTriageDetail(ai)	Renderiza card do 6o agente Triage.	AI_INSIGHTS.triage	Exibe urgent_reclassifications + aging_alerts
doRefresh()	Rebuilda todos os componentes apos mudanca de filtro.	Todos os build*	Respeita _builtPanels para evitar reconstrucao desnecessaria

4. Variáveis de Estado Global

O dashboard usa variáveis globais para rastrear estado de filtros e navegação. Definidas no início do script JS, atualizadas pelos event handlers.

Variável	Valor Inicial	Tipo	Descrição
selProject	null	String null	Projeto selecionado. null = Todos os Projetos
selYear	null	String null	Ano selecionado. null = todos
selMonth	null	String null	Mes selecionado. null = todos
activeSev	null	String null	Severidade da aba ativa. null = Hoje ou especial
activeView	'hoje'	String null	View especial ativa: hoje, attn, config, ai
_builtPanels	{}	Object	Cache de painéis já construídos por severidade
_mwCache	{}	Object	Cache de getMonthWin por chave sev+ano
_MEMO	{}	Object	Cache de getTix por ticket ID
charts	{}	Object	Instâncias Chart.js ativas por ID de canvas
_assState	{}	Object	Usuário selecionado por gráfico SEV-ASS
AI_INSIGHTS	Object	Object	JSONs dos 6 agentes embutidos no HTML
CFG	Object	Object	Configuração de SLA e ranges. Editável em ABA-CFG

4.1 ASSIGNED_STATUS — Status que contam como Assignado

```
var ASSIGNED_STATUS = {  
  'assigned_for_analysis':1,  
  'acknowledged':1,  
  'assigned_for_dev':1,  
  'waiting_for_prioritization':1  
};
```

Usado por buildAssigned() para filtrar tickets. Não respeita filtro de ano/mes — mostra estado atual da fila de trabalho.

5. Agente de IA — `smd_agent.py` v4

5.1 Arquitetura do Pipeline

O `smd_agent.py` é um script Python autônomo que executa 6 agentes em sequência, cada um recebendo o mesmo contexto JSON com os dados processados do CSV e retornando análise estruturada.

Etapa	Descrição
1 — Leitura CSV	pandas lê Tickets.csv com <code>sep=;</code> e encoding <code>utf-8-sig</code>
2 — Processamento	Calcula campos derivados: SLA, backlog, RC, monthly, daily
3 — Contexto JSON	Serializa tudo em JSON. Limita a 50k chars para caber no prompt Gemini
4 — Loop agentes	Para cada agente (ops, predictive, improvement, market, qa, triage):
4a — Prompt	Monta prompt com role + dados + schema JSON esperado + reasoning
4b — Gemini API	POST para <code>gemini-2.0-flash</code> com <code>temperature=0.1</code>
4c — Parseamento	Remove markdown, parse JSON, valida campos
5 — Salvamento	Salva <code>{agente}_output.json</code> em <code>C:/Dashboard/Resultados/</code>
6 — Log	Registra status no <code>smd_agent.log</code> (RotatingFileHandler 500KB x 3)

5.2 Configuração da API Gemini

Parâmetro	Valor
Modelo	<code>gemini-2.0-flash</code>
URL	<code>https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent</code>
Autenticação	API Key no parâmetro <code>?key=</code> da URL (sem headers extras)
Temperature	0.1 — baixa para respostas deterministas e estruturadas
Max output tokens	4096
Arquivo da chave	<code>C:/Dashboard/api_key.txt</code> — primeira linha não comentada
Custo	Gratuito — plano free do Google AI Studio
Limite diário	1.500 requests/dia
Limite por minuto	15 requests/minuto — script tem <code>sleep(1)</code> entre agentes

5.3 Schemas dos 6 Agentes

Cada agente recebe prompt com role específico + dados JSON + schema de resposta. Todos os schemas incluem o campo "reasoning" para TrustOps (M14).

Agente	Campos Obrigatorios no JSON de Resposta
ops	agent, timestamp, status, reasoning, executive_summary, health_score, sla_status, backlog_status, alerts[], recommendations[]
predictive	agent, timestamp, status, reasoning, executive_summary, volume_forecast, weekly_forecast[4], sla_risk, risk_events[], backlog_forecast
improvement	agent, timestamp, status, reasoning, executive_summary, maturity_level, quick_wins[], automation_opportunities[], kpi_suggestions[]
market	agent, timestamp, status, reasoning, executive_summary, market_position, benchmark, gaps[], trends[], roadmap
qa	agent, timestamp, status, reasoning, verdict, quality_score, executive_summary, sla_audit, findings[], data_quality, process_compliance[]
triage	agent, timestamp, status, reasoning, executive_summary, urgent_reclassifications[], aging_alerts[], priority_distribution, triage_score, recommendation

5.4 Execucao e Agendamento

```
# Todos os 6 agentes
python C:/Dashboard/smd_agent.py

# Agente especifico
python C:/Dashboard/smd_agent.py --agent triage

# Modo scheduler (07:00 e 15:00 diariamente)
python C:/Dashboard/smd_agent.py --schedule
```


6. Guia de Replicação Completo

Siga os passos abaixo para replicar o SMD em qualquer máquina Windows com Python 3.8+.

Passo 1 — Pre-requisitos

1. Python 3.8+ instalado (verificar: `python --version` no cmd)
2. pip install pandas (instalar biblioteca de análise de dados)
3. Google Chrome ou Edge para abrir o dashboard
4. Conta Google para acesso ao AI Studio (gratuito)

Passo 2 — Estrutura de Pastas

```
mkdir C:\Dashboard  
mkdir C:\Dashboard\input  
mkdir C:\Dashboard\Resultados
```

Passo 3 — Arquivo de Dados

- Exportar Tickets.csv do sistema de gestão de tickets
- Salvar em C:/Dashboard/input/Tickets.csv
- Verificar: separador ; (ponto-e-virgula) e encoding UTF-8
- Verificar: colunas obrigatórias listadas na seção 2.1

Passo 4 — Copiar os Arquivos

- Copiar `smd_agent.py` para C:/Dashboard/`smd_agent.py`
- Copiar `arrocha_dashboard_v6.html` para C:/Dashboard/

Passo 5 — Configurar API do Gemini (AI Studio)

5. Acessar aistudio.google.com e fazer login com conta Google
6. Clicar em "Get API Key" > "Create API key"
7. Copiar a chave (começa com Alza...)
8. Criar arquivo C:/Dashboard/`api_key.txt` e colar a chave (uma linha)

Passo 6 — Executar o Agente de IA

```
cd C:\Dashboard
```

```
python smd_agent.py
```

Aguardar: CONCLUIDO: 6 OK | 0 erro(s)

Passo 7 — Abrir o Dashboard

- Abrir C:/Dashboard/arrocha_dashboard_v6.html no Chrome
- Dashboard carrega com todos os dados do CSV
- Verificar: ABA-HOJ mostra a data atual
- Verificar: ABA-AI mostra 6 cards com resultados dos agentes

Passo 8 — Atualizar Dados Futuros

9. Substituir C:/Dashboard/input/Tickets.csv pelo novo CSV exportado
10. Regenerar o HTML executando o script Python de geracao
11. Executar smd_agent.py para atualizar os 6 insights de IA
12. Reabrir o arquivo HTML no browser

7. Registro de Bugs Corrigidos

Registro dos principais bugs identificados e corrigidos. Util para prevenir regressões em futuras modificações.

Bug	Causa Raiz	Correção
SLA vazio ao abrir	initProjects() auto-selecionava primeiro projeto sem dados SLA	Inicializar com selProject=null (Todos os Projetos)
SLA não filtrava por projeto	getSLAFiltered() sem filtro de selProject	Adicionar getTix(t.tid).prj === selProject
buildSLAMini conflito variável	Variável "html" conflitava com escopo externo	Renomear para "out" dentro da função
Tickets Assignados em branco	buildAssigned usava monthly.o_ids — perdia tickets de meses anteriores	Escanear _ROWS diretamente sem depender de monthly
buildKPI crash aba Hoje	activeSev=null causava D.summary[null] = undefined	Usar: _sev = activeSev "incident" como fallback
compCount ReferenceError	compCount era closure dentro de panelHTML	Promover compCount para função global antes de buildKPI
buildHoje em branco	activeView="hoje" não definido no init	Adicionar: activeSev=null; activeView="hoje" antes de doRefresh()
Range Progress sem mensagem	Faltava guard para sem filtro ativo	Exibir "Selecione..." quando selProject=null e selMonth=null
SLA urgency badge não aparecia	buildSLAMini não tinha lógica de urgência	Adicionar isUrgent check: pct < 80

8. Melhorias de Alta Prioridade Implementadas

As seguintes melhorias foram implementadas baseadas no documento Ecossistema SaaS ITSM Inteligente v3.0:

M2 — 6o Agente: Triage Analyst

Item	Detalhe
Arquivo modificado	smd_agent.py — AGENT_FILES + get_prompt("triage") + run_pipeline
Arquivo modificado	arrocha_dashboard_v6.html — AI_META + renderTriageDetail()
Output	C:/Dashboard/Resultados/triage_output.json
Exibe	Triage Score, urgent_reclassifications com seta P3→P1, aging_alerts
Onde ver	ABA-AI > card "Triage Analyst" (icone ►)

M14 — TrustOps: Botao Ver Raciocinio

Item	Detalhe
Arquivo modificado	smd_agent.py — campo "reasoning" adicionado a TODOS os 6 schemas
Arquivo modificado	arrocha_dashboard_v6.html — renderAIDetail() com botao expand
Comportamento	Botao "Ver raciocinio" aparece em cada card se campo reasoning presente
Onde ver	ABA-AI > qualquer card > clicar no card > botao azul-roxo

M5 — Previsao de Volume

Item	Detalhe
Arquivo modificado	smd_agent.py — campo weekly_forecast[4] adicionado ao schema predictive
Arquivo modificado	arrocha_dashboard_v6.html — buildHoje() chama buildHojeForecast()
Grafico	Historico 14 dias reais (azul) + projecao 4 semanas (roxo, opacidade por confianca)
Fallback	Se weekly_forecast ausente, estima com base na media dos ultimos 7 dias
Onde ver	ABA-HOJ > secao inferior "Previsao de Volume"

M3 — Alertas Proativos SLA

Item	Detalhe
Arquivo modificado	arrocha_dashboard_v6.html — buildSLAMini() + buildAttention()
Badge SEV-SLA	Ponto vermelho pulsante (CSS animation sla-pulse) quando pct < 80%
ABA-ATN	Alertas SLA calculados em runtime com getSLAFiltered() — mais atuais que estaticos
Threshold	80% de cumprimento = zona critica. Configuravel via CFG.sla
Onde ver	ABA-INC > SEV-SLA (badge) e ABA-ATN (alertas no topo)